Assignment 01 – Seven Segment Decoder

EIT HDL – Doku

Lukas Wolf

18. Oktober 2026

1 Aufgabe

Das Modul `sevenseg` übersetzt eine 4-Bit-Binärzahl `bin_i[3:0]` in das Segmentmuster `hex_o[6:0]` einer Siebensegmentanzeige. Zusätzlich liefert `hexn_o[6:0]` das negierte Muster für Low-aktive Anzeigen.

2 Implementierung

Der Decoder ist als Verhaltensbeschreibung in einem `always_comb`-Block mit einer `case`-Anweisung umgesetzt. Am Ende wird `hexn_o` negiert.

```
always_comb begin
    case (bin_i)
        4'h0 : hex_o = 7'b0111111;
        4'h1 : hex_o = 7'b0000110;
        ...
        4'hF : hex_o = 7'b1110001;
        default: hex_o = 7'b0000000;
    endcase
end

assign hexn_o = ~hex_o;
```

3 Verifikation

Die Testbench legt alle 16 Eingangswerte im Abstand von 100 ns an, gibt Ein- und Ausgänge mit `$display` in der Konsole aus. Die Ausgaben werden anschließend mit Figure 3 der Angabe verglichen. Gestartet wird die Simulation in ModelSim mit `do sim_tb_sevenseg.tcl`. Das Skript öffnet das Wave-Fenster mit allen relevanten Signalen (Testbench-Signale, DUT-Ports und die Einzelbits von hex_o).

Simulator-Ausgabe

```
bin_i = 0 | hex_o = 0111111 | hexn_o = 1000000
bin_i = 1 | hex_o = 0000110 | hexn_o = 1111001
bin_i = 2 | hex_o = 1011011 | hexn_o = 0100100
bin_i = 3 | hex_o = 1001111 | hexn_o = 0110000
bin_i = 4 | hex_o = 1100110 | hexn_o = 0011001
bin_i = 5 | hex_o = 1101101 | hexn_o = 0010010
bin_i = 6 | hex_o = 1111101 | hexn_o = 0000010
bin_i = 7 | hex_o = 0000111 | hexn_o = 1111000
bin_i = 8 | hex_o = 1111111 | hexn_o = 0000000
bin_i = 9 | hex_o = 1101111 | hexn_o = 0010000
bin_i = a | hex_o = 1110111 | hexn_o = 0001000
bin_i = b | hex_o = 1111100 | hexn_o = 0000011
bin_i = c | hex_o = 0111001 | hexn_o = 1000110
bin_i = d | hex_o = 1011110 | hexn_o = 0100001
```

```
bin_i = e | hex_o = 1111001 | hexn_o = 0000110
bin_i = f | hex_o = 1110001 | hexn_o = 0001110
```

Wave-Fenster

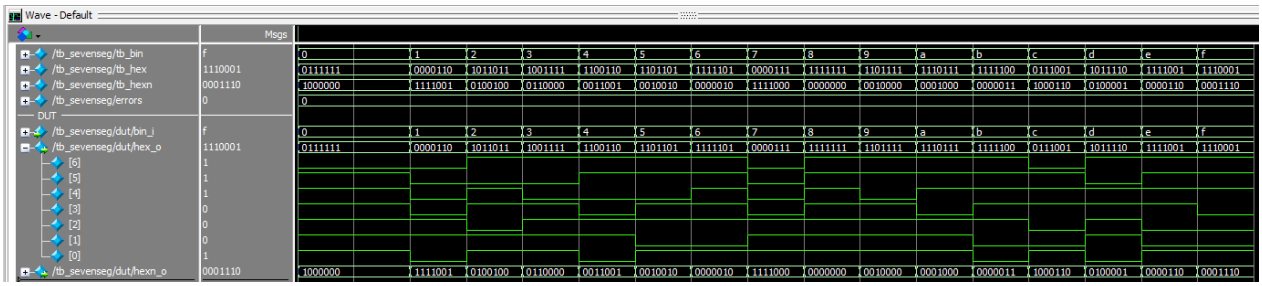


Abbildung 1: ModelSim Wave-Fenster der Simulation von `tb_sevenseg`.

Ergebnis Alle 16 Eingangskombinationen liefern das erwartete Muster, hexn_o ist in jedem Fall die bitweise Negation von hex_o. Das Modul erfüllt damit die Spezifikation.

4 Diskussion der Implementierungsart

Methode	Vorteile	Nachteile
Gatter (`and`, `or`, ...)	Man bestimmt jedes einzelne Gatter selbst.	Für 7 Segmente braucht man sehr viele Gatter. Das ist viel Arbeit, man macht leicht Fehler und der Code ist schwer zu lesen.
`assign` mit Formeln	Kurzer Code.	Schwierig zu lesen und nachzuvollziehen.
`always_comb` mit `case` (gewählt)	Code ist leicht zu lesen und zu ändern.	Etwas mehr Zeilen Code.